

Progress Report 2

GPULAB: High Performance Computing with Graphic Cards
Project Topic 1-2: CT Volume Reconstruction

Sanan Akther, Divyansh Sharma, Joseph Maher Labib Estawro

July 15, 2026

Contents

1 Progress Summary

Since Progress Report 1, all four reconstruction modes have been fully implemented, debugged, and validated. The CPU C port, GPU buffer mode, GPU image-object mode, and GPU optimized mode all produce numerically correct output with Mean Squared Error (MSE) below 2×10^{-8} relative to the CPU reference on the 256^3 dataset.

This report covers the C CPU implementation (Sections ??-??), the three GPU OpenCL modes (Section ??), preliminary performance results (Section ??), and a correctness validation (Section ??). Remaining work (512^3 dataset, final optimization analysis, convergence plots) is deferred to the final report.

2 CPU C Implementation

2.1 Forward Projection (fp_cpu)

The forward projection ray-marches through the 3D volume for each detector pixel and angle. The outer loop is parallelized over $N_\theta \times W$ work items using OpenMP `collapse(2)`, giving $75 \times 512 = 38,400$ independent tasks:

```
1 #pragma omp parallel for collapse(2) schedule(static)
2 for (int ip = 0; ip < np; ip++) {
3     for (int iu = 0; iu < W; iu++) {
4         /* inner loop: ray march through Ny=512 samples */
5         for (int iv = 0; iv < H; iv++) { ... }
6     }
7 }
```

Per-sample trilinear interpolation uses a single unsigned bounds check (avoids branching) and incremental ray stepping (avoids one multiply per sample):

```
1 if ((unsigned)x0 < (unsigned)(Nxz-1) &&
2     (unsigned)y0 < (unsigned)(Ny-1) &&
3     (unsigned)z0 < (unsigned)(Nxz-1)) {
4     /* 8-tap trilinear */
5 }
6 wx += dox; wy += doy; wz += doz; /* incremental step */
```

Result: ≈ 5.8 s/epoch (was ≈ 28 s in the naive implementation).

2.2 Backprojection (bp_cpu)

Backprojection is parallelized over (ix, iy) voxel pairs with `collapse(2)`. Each pair owns a unique z-strip of the output volume so no atomics or reduction buffers are needed. A precomputed cos/sin LUT eliminates per-voxel transcendental calls:

```
1 #pragma omp parallel for collapse(2) schedule(static)
2 for (int ix = 0; ix < Nxz; ix++) {
3     for (int iy = 0; iy < Nxz; iy++) {
4         float *strip = volume + VOL_IDX(Nxz-1-ix, Nxz-1-iy, 0, Nxz, Ny);
5         for (int ip = 0; ip < np; ip++) {
6             float ca = ca_lut[ip], sa = sa_lut[ip];
7             /* bilinear lookup into projection slice */
8         }
9         /* scale strip in-place */
10    }
11 }
```

Result: $\approx 1.3\text{s/epoch}$. Total CPU: $\approx 7.3\text{s/epoch}$.

2.3 MLEM Reconstruction Loop

The iterative update follows the MLEM rule from the Python reference:

$$v^{(k+1)} = v^{(k)} \cdot \frac{\mathcal{B}\left(\frac{p_0}{\mathcal{F}(v^{(k)})}\right)}{\mathcal{B}(\mathbf{1})}$$

A critical subtlety: the Python `bp_func` applies the cone-weight correction *inside* each backprojection call. Consequently, the cone weight must be applied to the ratio $p_0/\mathcal{F}(v)$ *before* backprojection each epoch, and $\mathcal{B}(\mathbf{1})$ must be computed from cone-weighted ones. Applying cone weight once to the measured data p_0 (as a preprocessing step) is incorrect and produces divergence.

3 Implementation Challenges

3.1 Projection Data Layout

The Python reference transposes and flips the projection data inside `bp_func`:

```
proj[:, ::-1, :].transpose(0,2,1) / voxelSize
```

This converts the HDF5 layout $[N_\theta][H][W]$ to $[N_\theta][W][H]$ with a flipped H-axis, then scales by $1/\delta_v$. This preprocessing must be applied to both the ratio and the all-ones buffer before every backprojection call. A dedicated `preprocess_proj` OpenCL kernel handles this for GPU modes.

3.2 Forward Projection Boundary Condition

The OpenCL image3D sampler with `CLK_ADDRESS_CLAMP_TO_EDGE` extends edge voxels beyond the volume boundary, artificially inflating forward projection values. For an all-ones volume, the GPU returned `max=6.14` while the CPU returned `max=2.84` — a $2.16\times$ mismatch that caused divergence at every epoch.

The fix is an explicit bounds check in the `fp_image` kernel inner loop, skipping any sample whose fractional voxel index falls outside $(-0.5, \text{dim} - 0.5)$:

```
1 if (xi >= 0.f && xi < (float)(Nxz - 1) &&
2    yi >= 0.f && yi < (float)(Ny - 1) &&
3    zi >= 0.f && zi < (float)(Nxz - 1)) {
4    float4 coord = (float4)(zi+0.5f, yi+0.5f, xi+0.5f, 0.f);
5    float density = read_imagef(volume_img, vol_samp, coord).x;
6    val += density * dt * rd_norm;
7 }
```

3.3 Backprojection of All-Ones Scale

In GPU image mode, `bp_ones` was initially computed from a raw all-1.0 image with no preprocessing applied. The resulting scale was off by $1/\delta_v \approx 313\times$, causing the MLEM denominator to be wrong. The fix copies a preprocessed (cone-weighted + flip/transpose $/\delta_v$) all-ones buffer into the image object via `clEnqueueCopyBufferToImage` before running backprojection.

4 GPU OpenCL Implementation

4.1 Mode Overview

Three GPU modes are implemented, all sharing the same `proj_divide`, `vol_update`, `cone_weight_hw`, and `preprocess_proj` utility kernels:

Mode	Projection storage	Volume storage	Interpolation
gpu-buf	cl_mem buffer	cl_mem buffer	Manual bilinear/trilinear
gpu-img	image2d_array_t	image3d_t	Hardware sampler
gpu-opt	image2d_array_t	image3d_t	Hardware sampler + LUT

Table 1: GPU implementation modes

4.2 Buffer Mode (gpu-buf)

Work-item mapping: one item per voxel (ix, iy, iz) for backprojection; one item per detector pixel (iu, iv, ip) for forward projection. Manual bilinear and trilinear interpolation in the kernel. Global work size: 256^3 for bp, $512 \times 512 \times 75$ for fp.

4.3 Image Mode (gpu-img)

Projections are stored as a `CL_MEM_OBJECT_IMAGE2D_ARRAY` with hardware bilinear sampling in the bp kernel. The volume is stored as a `CL_MEM_OBJECT_IMAGE3D` with hardware trilinear sampling in the fp kernel. This eliminates manual interpolation code and exploits the GPU texture cache. The image layout uses width = H , height = W for the projection array (matching the preprocessed $[N_\theta][W][H]$ layout) and width = N_y , height = N_{xz} , depth = N_{xz} for the volume.

4.4 Optimized Mode (gpu-opt)

Adds a precomputed float2 cos/sin LUT for the angle loop in backprojection, replacing one `cos()` and one `sin()` call per voxel per angle with a single coalesced global memory read:

```
1 float2 cs = angle_cs[ip];    /* preloaded on host: {cos(angle), sin(angle)
   } */
2 float ca = cs.x, sa = cs.y;
```

For $N_{xz}^2 \times N_y \times N_\theta = 256^2 \times 256 \times 75 \approx 1.26 \times 10^9$ operations per epoch, eliminating transcendental calls reduces bp time from ≈ 0.05 s to ≈ 0.04 s (limited by texture bandwidth).

Work-group sizes are tuned to $8 \times 8 \times 4 = 256$ threads for bp kernels (matching the Hawaii GPU wavefront size of 64, occupying 4 wavefronts per group) and $16 \times 16 \times 1 = 256$ threads for fp kernels.

5 Performance Results

All timings on AMD Hawaii (Radeon R9 390), 256^3 volume, 512×512 detector, 75 projection angles, 10 epochs.

The dominant cost in `gpu-img` and `gpu-opt` is forward projection (~ 0.12 s/epoch), which performs $\sim 2.5 \times 10^9$ trilinear lookups per epoch via the image3D hardware sampler. Backprojection costs ~ 0.04 s/epoch.

Mode	Time/epoch	Total (10 ep)	Speedup vs CPU
cpu	~ 7.3 s	75.9 s	$1\times$
gpu-buf	~ 1.0 s	11.9 s	$\sim 7\times$
gpu-img	~ 0.157 s	1.70 s	$\sim 47\times$
gpu-opt	~ 0.147 s	1.55 s	$\sim 50\times$

Table 2: Reconstruction time, 256^3 dataset, 10 epochs

6 Correctness Validation

All GPU modes are validated against the CPU reference by MSE over the full $256^3 = 16,777,216$ -voxel volume after 10 epochs:

Mode	min	max	mean	MSE vs CPU
cpu	0.0000	0.9835	0.0066	(reference)
gpu-buf	0.0000	0.9838	0.0066	1.6×10^{-8}
gpu-img	0.0000	0.9836	0.0066	1.9×10^{-8}
gpu-opt	0.0000	0.9837	0.0066	1.9×10^{-8}

Table 3: Validation results, 256^3 dataset, 10 epochs

The residual MSE ($\sim 10^{-8}$) is consistent with float32 rounding differences between manual trilinear (CPU/buffer) and hardware sampler (image modes). All modes agree on the reconstructed mean and remain strictly positive, confirming MLEM convergence is correct.

7 Remaining Work

The following items are planned for the final report:

1. **512³ dataset:** Run all modes on `proj_512.75.hdf5` and report scaled performance and memory usage (~ 512 MB volume).
2. **Convergence analysis:** Plot reconstructed volume statistics vs. epoch number for all modes; compare convergence rate GPU vs. CPU.
3. **Optimization analysis:** Quantify individual contributions of the LUT, work-group tuning, and image sampler vs. buffer interpolation.
4. **Final MSE table:** Full 100-epoch run for all modes with MSE and max-diff reported.

8 Summary

All four reconstruction modes are implemented, validated, and working correctly. The GPU optimized mode achieves a **50 $\times$** speedup over the CPU reference on the 256^3 dataset with MSE $< 2 \times 10^{-8}$. The main technical challenges were the cone-weight ordering, the `fp_image` boundary inflation from `CLAMP_TO_EDGE`, and the `bp_ones` preprocessing scale. All three are resolved. The final report will cover the 512^3 dataset and a full convergence and optimization analysis.